GATE9 — Live Production Cache Validation Batch

Framework: 18 — AI Management (36 measures per company) **Date:** 2026-10-17 (UTC) **Operator:** Andy Howard (Schroders) **Scope:** Framework-level validation of the layered cache in the *live production* pipeline. No company- or topic-specific changes.

---

1. Objective

Confirm, against the **real production worker with the cache layer enabled**, that:

1. The cache-on code deployed to production is *operable in-situ* — i.e. the production worker can claim and complete a full re-scoring batch end-to-end with `CACHE_LAYER_POLICY=on`.
2. Measure verdicts produced under the live cache-on pipeline can be compared, cell-by-cell, against a pre-cache baseline, producing a verdict-flip table (Yes→No / No→Yes / Total).

Money spend for this batch was explicitly pre-approved by the user.

---

2. What was run

Item	Value
Replay batch	**1290** (sourced from baseline batch 1288)
Companies	10 (company_id 9–18)
Cells per batch	360 (10 companies × 36 measures)
Jobs	10/10 **completed**, 0 failed
Executed by	**Production worker** (service `27920e1f`), cache-on, deployed SHA `748a506`
Enqueue method	`a5_replay_enqueue.ts` (canary 1 + rest 9); operator only *enqueued* — the prod worker claimed→completed every job
Replay mode	`skipFetch=true` (frozen corpus; discovery/byte-fetch bypassed)

Deployment-target verification (constraint: code tested == code in production):

Batch	git SHA (from `analysis_results.manifest.gitSha`)	Role
1290 (replay,		**Exact deployed**

cache-on)	`748a506542a3e4b837ff792b846171bbaf5a5ffe`	**production SHA**
1288 (baseline)	`f15f8b69bafdab133a09a92918dc847303b6b91f`	Historical pre-cache reference

Batch 1290 ran on the identical SHA that is live in production (app + worker both verified on `748a506`, with `CACHE_LAYER_POLICY=on` and `DISCOVERY_PARALLEL_POLICY=on` set on both services). `f15f8b6` is a confirmed git ancestor of `748a506`.

---

3. Verdict-flip result

3.1 Overall (MANDATORY Yes→No / No→Yes / Total columns)

Cells compared	Unchanged	Yes→No	No→Yes	Total flips
360	343	**7**	**10**	**17**

3.2 Per-company

company_id	Company	Yes→No	No→Yes	Total flips
9	CSX Corporation	0	0	**0**
10	MAGNA INTERNATIONAL INC.	1	1	**2**
11	THE SOUTHERN COMPANY	1	1	**2**
12	S&P GLOBAL INC.	1	1	**2**
13	MARTIN MARIETTA MATERIALS, INC.	1	0	**1**
14	DARDEN RESTAURANTS, INC.	1	0	**1**
15	BCE INC.	2	4	**6**
16	STRYKER CORPORATION	0	2	**2**
17	AMPHENOL CORPORATION	0	0	**0**
18	AFLAC INCORPORATED	0	1	**1**
Total		**7**	**10**	**17**

Source of verdicts: table `analysis_results` (one immutable `results_data` snapshot per batch), not `measure_scores` — see §5.

---

4. Attribution — the 17 flips are a CONFOUNDED comparison, NOT a clean cache measurement

This is the central honesty point. The 17 flips **must not** be read as "cache caused 17 verdict changes." They are the *difference between two independent fresh-scoring runs made at different times on different code*, and the cache contributed no verdict-altering reuse in this batch.

4.1 ESTABLISHED

1. **Cache enabled in production** — `CACHE_LAYER_POLICY=on` verified on both app and worker services.
2. **In-situ operability** — the production worker, on the deployed cache-on SHA `748a506`, re-scored 10 companies × 36 measures with **0 failures**. The operator only enqueued; production claimed and completed every job. This is the primary thing this batch proves.
3. **Deterministic cache-neutrality (offline, prior gates)** — GATE4/6/7/8 already established that cache cold ≡ warm produces byte-identical verdicts (flips 0/0/0) with warm-hit speedups. See §6.

4.2 CONFOUNDS on the 17-flip live comparison

- **Cross-timepoint LLM re-score non-determinism.** 1288 was scored 2026-10-16; 1290 was scored 2026-10-17. Two independent fresh-scoring runs on the same frozen corpus will differ from LLM stochasticity alone, with no cache involved.
- **Code delta** `f15f8b6 → 748a506` (not a same-SHA control). Runtime files changed: `cache-layers.ts` (+457, new), `pipeline-cache.ts` (+200, new), `discovery-parallel.ts` (+497, new), `analyzer.ts` (+58), `discovery.ts` (+119), `pipeline.ts` (+23). With `skipFetch=true`, `discovery.ts` / `discovery-parallel.ts` do not execute (discovery bypassed).
- **Cold index on first replay.** The replay uses `skipFetch=true`, so the BYTES seam (`cacheDocumentFetch`) is never exercised. Only the INDEX seam (`cacheRetrievalIndex`) could be — and on the *first* replay of this corpus with cache-on, the index cache is COLD (miss → compute → store, no warm reuse). **The cache therefore provided no verdict-altering reuse in this batch.** The 17 flips are consistent with baseline re-score variance, not cache-induced divergence.
- **No simultaneous same-SHA cache-off control run** was performed (a clean isolation would require same-SHA cache-on vs cache-off on the same corpus at the same time).

4.3 MISSING

- **In-situ warm-hit evidence** was not captured in this batch (index cache cold on first replay). Capturing it would require a **second replay** of batch 1290's now-cached corpus — i.e. a "Trial 2" — which needs explicit user approval and additional spend. **Not run** (standing constraint). Offline warm-hit evidence (GATE7/GATE8) stands in its place.

4.4 CONTRADICTED

- None. No evidence contradicts cache verdict-neutrality; the deterministic offline gates affirmatively support it.

4.5 Bottom line

> The live batch **confirms the production cache is operable in-situ** (full 10×36 re-score, 0 failures, on the exact deployed cache-on SHA). It does **not** constitute a clean cache-isolation measurement: the 17 flips are a confounded cross-timepoint, cross-code re-score delta, and with `skipFetch=true` + a cold index the cache performed no verdict-altering reuse. Deterministic cache verdict-neutrality was already proven offline (GATE4/6/7/8, flips 0/0/0).

---

5. Discovery — baseline `measure_scores` were overwritten by the replay

`measure_scores` is keyed per **(company, framework)**, not per batch. `replaceMeasureScoresFenced` (`server/storage.ts`) does a framework-scoped `delete where company_id AND framework_id` then insert. Re-scoring companies 10–18 therefore **deleted their batch-1288 `measure_scores` rows** (reassigned to 1290). After the run, `measure_scores` for batch 1288 = 0 rows (was 324); batch 1290 = 360 rows.

Recovery path (used for every number above): the per-batch verdicts are preserved immutably in table **`analysis_results`** (one `results_data` jsonb snapshot per batch, `immutable_snapshot` flag, never overwritten). Both 1288 and 1290 retain full 10-company × 36-measure snapshots there, including company 9 (which never had a `measure_scores` baseline row). The flip table was computed from these snapshots.

Operational note (framework-level): per-batch verdict comparisons must be sourced from `analysis_results`, not `measure_scores`, because any later re-score of the same (company, framework) overwrites the latter. This is a general property of the storage layer, not specific to these companies.

---

6. Prior (offline) cache-neutrality evidence referenced above

Gate	Setup	Flips (Y→N / N→Y / Total)	Warm-hit evidence
GATE4	cold vs warm, single	0 / 0 / 0	cold ≡ warm byte-identical
GATE6	serial vs parallel discovery	0 / 0 / 0	—
GATE7	30 docs, bytes	0 / 0 / 0	30/30 100% warm-hit,

	seam		byte-identical
GATE8	end-to-end	0 / 0 / 0	cold 94.68s/147 calls/$0.3873 vs warm 10.47s/1 call/$0.0001 (~9×)

(Figures as recorded in the respective gate reports; re-verify against those files before re-quoting.)

---

7. Verdict

- **In-situ cache operability:** ✅ ESTABLISHED (10/10 jobs, 0 failures, exact deployed cache-on SHA `748a506`).
- **Clean cache-isolation of the 17 flips:** ❌ NOT established — confounded (cross-timepoint, cross-code, cold index, no same-SHA control). INCONCLUSIVE as a cache measurement, not a failure.
- **Deterministic cache verdict-neutrality:** ✅ ESTABLISHED offline (GATE4/6/7/8, flips 0/0/0).
- **In-situ warm-hit:** ☐ MISSING by design this batch (cold index); would require an approved Trial 2.

---

8. Reproduction

Flip query (run against production DB, read-only):

```
        with flat as (
          select ar.batch_id, (co->>'companyId')::int company_id, (co-
>>'companyName') name,
                 (ms->>'measureId') measure_id, (ms->>'verdict') verdict
          from analysis_results ar,
               jsonb_array_elements(ar.results_data) co,
               jsonb_array_elements(co->'measureScores') ms
          where ar.batch_id in (1288,1290)),
        base as (select * from flat where batch_id=1288),
        repl as (select * from flat where batch_id=1290)
        select b.company_id, max(b.name) company,
          count(*) filter (where b.verdict='Yes' and r.verdict='No') y_to_n,
          count(*) filter (where b.verdict='No'  and r.verdict='Yes')
n_to_y,
          count(*) filter (where b.verdict is distinct from r.verdict)
total_flips
        from base b join repl r using (company_id, measure_id)
        group by b.company_id order by b.company_id;
```